Building Agents with OpenAI’s AgentKit and Responses API

Lucas Soares

O'Reilly Live Training

Table of Contents

1. The Problem: Why Layers Matter

Table of Contents

1. The Problem: Why Layers Matter

2. Layer 1: Responses API — the primitive

Table of Contents

1. The Problem: Why Layers Matter

2. Layer 1: Responses API — the primitive

3. Layer 2: Agents SDK — the framework

Table of Contents

1. The Problem: Why Layers Matter

2. Layer 1: Responses API — the primitive

3. Layer 2: Agents SDK — the framework

4. Layer 3: AgentKit — the product platform

Table of Contents

1. The Problem: Why Layers Matter

2. Layer 1: Responses API — the primitive

3. Layer 2: Agents SDK — the framework

4. Layer 3: AgentKit — the product platform

5. When to Use Each Layer

The Problem

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions
- **Multi-agent routing** — delegating tasks to specialized agents

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions
- **Multi-agent routing** — delegating tasks to specialized agents
- **Guardrails** — input/output validation, safety checks

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions
- **Multi-agent routing** — delegating tasks to specialized agents
- **Guardrails** — input/output validation, safety checks
- **Observability** — tracing, debugging, cost monitoring

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions
- **Multi-agent routing** — delegating tasks to specialized agents
- **Guardrails** — input/output validation, safety checks
- **Observability** — tracing, debugging, cost monitoring
- **User Interface** — a chat experience your users actually interact with

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions
- **Multi-agent routing** — delegating tasks to specialized agents
- **Guardrails** — input/output validation, safety checks
- **Observability** — tracing, debugging, cost monitoring
- **User Interface** — a chat experience your users actually interact with

OpenAI's answer: a layered stack — each layer builds on the previous one.

The Three-Layer Stack

1

Responses API

The Primitive
Direct API calls
Built-in tools & state

→

2

Agents SDK

The Framework
Multi-agent orchestration
Guardrails & tracing

→

3

AgentKit

The Product Platform
Agent Builder + ChatKit
Connectors + Evals

The Three-Layer Stack

Each layer builds on the previous — you pick the level of abstraction you need.

Layer 1: Responses API

The Primitive

Responses API — What & Why

Released **March 2025**. Replaces both Chat Completions API and Assistants API.

Responses API — What & Why

Released **March 2025**. Replaces both Chat Completions API and Assistants API.

Key Innovations

- **Items-based I/O** — polymorphic output (text, tool calls, reasoning)
- **Server-side state** via `previous_response_id`
- **Reasoning preservation** across turns

Built-in Tools

- `web_search_preview`
- `file_search`
- `code_interpreter`
- `computer_use_preview`
- `mcp` (Model Context Protocol)

Responses API — What & Why

Released **March 2025**. Replaces both Chat Completions API and Assistants API.

Key Innovations

- **Items-based I/O** — polymorphic output (text, tool calls, reasoning)
- **Server-side state** via `previous_response_id`
- **Reasoning preservation** across turns

Built-in Tools

- `web_search_preview`
- `file_search`
- `code_interpreter`
- `computer_use_preview`
- `mcp` (Model Context Protocol)

This is what we've been using in the notebooks so far.

[1] Why we built the Responses API

Layer 2: Agents SDK

The Framework

Agents SDK — Overview

Open-source framework. **Python + TypeScript**.

```
pip install openai-agents
```

Agents SDK — Overview

Open-source framework. **Python + TypeScript**.

```
pip install openai-agents
```

The 5 Core Primitives

Agents SDK — Overview

Open-source framework. **Python + TypeScript**.

```
pip install openai-agents
```

The 5 Core Primitives

Works with **100+ LLMs** — not locked to OpenAI.

[1] openai-agents-python (GitHub)
[2] Agents SDK Documentation

Layer 3: AgentKit

The Product Platform

AgentKit — The Full Platform

Launched at **DevDay, October 2025**. No platform fee — standard API pricing.

AgentKit — The Full Platform

Launched at **DevDay, October 2025**. No platform fee — standard API pricing.

Agent Builder

Visual drag-and-drop canvas for composing multi-agent workflows

ChatKit

Embeddable chat UI — React components + Web Component

Connector Registry

Pre-built data connectors — Dropbox, Google Drive, SharePoint, MCP

Evals for Agents

Datasets, trace grading, automated prompt optimization

[1] Introducing AgentKit — OpenAI

ChatKit — Embeddable Chat UI

Frontend-only — React components + framework-agnostic Web Component.

ChatKit — Embeddable Chat UI

Frontend-only — React components + framework-agnostic Web Component.

```
import { ChatKit, useChatKit } from '@openai/chatkit-react';

const chatkit = useChatKit({
  api: { getClientSecret },
  .....

return <ChatKit control={chatkit.control} />;
```

ChatKit — Embeddable Chat UI

Frontend-only — React components + framework-agnostic Web Component.

```
import { ChatKit, useChatKit } from '@openai/chatkit-react';

const chatkit = useChatKit({
  api: { getClientSecret },
  .....

return <ChatKit control={chatkit.control} />;
```

Includes: file upload, light/dark theming, tool invocation display, chain-of-thought visualization, suggestion chips, event system, and more.

[1] ChatKit — OpenAI Docs
[2] chatkit-js (GitHub)

ChatKit — Backend Token Exchange

```
// POST /api/create-session (your backend)
const response = await fetch(
  "https://api.openai.com/v1/chatkit/sessions",
  {
    method: "POST",
    headers: {
      ....
);

const { client_secret } = await response.json();
// Return client_secret to the frontend
```

ChatKit — Backend Token Exchange

```
// POST /api/create-session (your backend)
const response = await fetch(
  "https://api.openai.com/v1/chatkit/sessions",
  {
    method: "POST",
    headers: {
      ....
);

const { client_secret } = await response.json();
// Return client_secret to the frontend
```

Key pattern: Your server-side API key is exchanged for a short-lived client token. Per-user sessions via cookies. The frontend *never* sees your API key.

Demo: ChatKit App

When to Use Each Layer

Direct API Control

- **Responses API** — direct calls, full control, learning

Multi-Agent Systems

- **Agents SDK** — orchestration, custom logic, guardrails

Ship a Chat UI

- **ChatKit** — embed a chat experience fast

Visual Workflows & Evals

- **Agent Builder** — drag-and-drop, non-dev teams
- **Evals** — measure and improve agent quality

Key Takeaways

Key Takeaways

- **Start with Responses API** for maximum control and the deepest learning

Key Takeaways

- **Start with Responses API** for maximum control and the deepest learning
- **Add Agents SDK** when you need multi-agent orchestration, guardrails, tracing

Key Takeaways

- **Start with Responses API** for maximum control and the deepest learning
- **Add Agents SDK** when you need multi-agent orchestration, guardrails, tracing
- **Use AgentKit / ChatKit** when you're ready to ship to real users

Key Takeaways

- **Start with Responses API** for maximum control and the deepest learning
- **Add Agents SDK** when you need multi-agent orchestration, guardrails, tracing
- **Use AgentKit / ChatKit** when you're ready to ship to real users
- Each layer **builds on the previous** — no vendor lock-in at the SDK level

Key Takeaways

- **Start with Responses API** for maximum control and the deepest learning
- **Add Agents SDK** when you need multi-agent orchestration, guardrails, tracing
- **Use AgentKit / ChatKit** when you're ready to ship to real users
- Each layer **builds on the previous** — no vendor lock-in at the SDK level
- The whole stack is available at **standard API pricing** — no platform fee

What's Next

We continue building with the **Responses API** in the next sections:

What's Next

We continue building with the **Responses API** in the next sections:

- **Paper data extraction** — structured outputs from PDFs

What's Next

We continue building with the **Responses API** in the next sections:

- **Paper data extraction** — structured outputs from PDFs
- **Chat with paper agent** — document Q&A with tool use

What's Next

We continue building with the **Responses API** in the next sections:

- **Paper data extraction** — structured outputs from PDFs
- **Chat with paper agent** — document Q&A with tool use
- **Video script generation** — multi-modal with images

What's Next

We continue building with the **Responses API** in the next sections:

- **Paper data extraction** — structured outputs from PDFs
- **Chat with paper agent** — document Q&A with tool use
- **Video script generation** — multi-modal with images
- **Research report generation** — end-to-end agentic workflow

What's Next

We continue building with the **Responses API** in the next sections:

- **Paper data extraction** — structured outputs from PDFs
- **Chat with paper agent** — document Q&A with tool use
- **Video script generation** — multi-modal with images
- **Research report generation** — end-to-end agentic workflow

The patterns you learn here apply at every layer of the stack.

Q&A & References

- Why we built the Responses API — OpenAI Blog
- Responses API Reference — OpenAI Docs
- openai-agents-python — GitHub
- Agents SDK Documentation — OpenAI
- Introducing AgentKit — OpenAI
- ChatKit Guide — OpenAI Docs
- chatkit-js — GitHub

Q&A & References

- Why we built the Responses API — OpenAI Blog
- Responses API Reference — OpenAI Docs
- openai-agents-python — GitHub
- Agents SDK Documentation — OpenAI
- Introducing AgentKit — OpenAI
- ChatKit Guide — OpenAI Docs
- chatkit-js — GitHub

Questions before we move on?